MIRAI BANK PROJECT

Milestone１ [C1]

Prototype Basic Specifications Document

Table of Contents

1. Purpose of This Document 3
2. Service Concept and Basic Configuration 4
   2.1 Service Concept 4
   2.2 Four-Layer Conceptual Model 5
   2.3 Capsule Lifecycle (Status Transitions) 6
   2.4 Three-Role Permission Model 6
3. System Configuration (Corresponding to: C1-1) 7
   3.1 Technology Stack 8
   3.2 Cardano-Specific Technology Stack 8
   3.3 Key Points of the Data Model 11
4. Linking Blockchain Functions and the User Interface (Corresponding to: C1-2) 12
5. Group Capsule Opening Modes (D-11) 16
   5.1 The Three Opening Modes 16
   5.2 Extending the Data Model 17
   5.3 How an Opening Is Decided 18
   5.4 Protecting Mode ③ (Emergency Opening) 19
6. Summary 21

1. Purpose of This Document

This document is Milestone 1 ("Prototype research and basic specifications") Output C1, the Prototype Basic Specifications Document. Building on the previously organized Outputs A1 (NFT standard and decentralized storage selection) and B1 (UI operations list and data flow), it defines the following two points.

- **C1-1:**
  System configuration (architecture, technology stack, data model)
- **C1-2:**
  Five ways in which blockchain functions and the user interface are linked

In addition, this document clarifies the Mirai project's basic configuration concepts (service concept, four-layer model, permission model) that underlie the system configuration, forming the basis for discussion in the next milestone, M2 (Prototype Detailed Design). The actual implementation approach (specific components, detailed processing steps, etc.) will be addressed in M2; this document focuses as much as possible on plain-language explanations.

2. Service Concept and Basic Configuration

2.1 Service Concept

MIRAI BANK TIME Capsule (ChainCapsule) is a digital time-capsule service, built on the Cardano blockchain, centered on the concept of "accurately recording people's thoughts for the future." It authenticates users by their Cardano wallet address, and encrypts, seals, and stores content such as text and images until a future date specified by the user. When a capsule is finalized, a CIP-25-compliant NFT is issued and recorded on-chain in a tamper-proof form, guaranteeing the authenticity of the recorded information and the right to open it.

The envisioned use cases range broadly, from an individual's own record (a letter to oneself) to a school class's graduation capsule, a wedding's bridal capsule, or a company's anniversary capsule, and the design supports both individual and group use.

2.2 Four-Layer Conceptual Model

At the core of the service is a four-layer conceptual model, starting from the individual user: "Individual User → Group → Capsule → Piece." Both individual and group capsules are handled uniformly, internally, through the same Group table.

Layer	Concept	Description	Cardinality
L1	Individual User	A Cardano wallet address (addr1...) serves as the user ID. No password is required. This is the starting point for every operation.	1 user → N group memberships
L2	Group	There are two types: a hidden group automatically prepared for individual use (SOLO), and a group explicitly created when multiple people are involved (REAL).	1 group → N capsules
L3	Capsule (Project)	Once finalized (sealed), a certificate (NFT) is issued and recorded on the blockchain. Status moves through DRAFT → LOCKED → OPENED.	1 capsule → N pieces
L4	Piece	The smallest unit of content (text or an image). Encrypted on the device. For a group, a registration slot is automatically prepared for every member.	1 capsule → N pieces (1 or more)

Table 2-1 The Four-Layer Conceptual Model (L1 to L4)

An individual's capsule is managed uniformly, internally, as an automatically generated personal group, so the user need not be aware of the concept of a group at all. On the other hand, in cases involving multiple people, such as a school class or a company department, the group's manager explicitly creates a group.

2.3 Capsule Lifecycle (Status Transitions)

Status	Color	Description	Transition Condition
DRAFT	Editable	Being created; supports all CRUD operations	Set automatically at creation
LOCKED	Sealed	The certificate (NFT) has been issued and the content can no longer be changed	Manager performs the finalize action and signs with their wallet
OPENED	Opened	The opening date/time has arrived and the content can be viewed	Opening date/time reached → opening process executed

Table 2-2 Capsule Status Transitions

2.4 Three-Role Permission Model

Permissions within a group are managed through three roles. For an individual's capsule, the person who created it automatically holds all three roles.

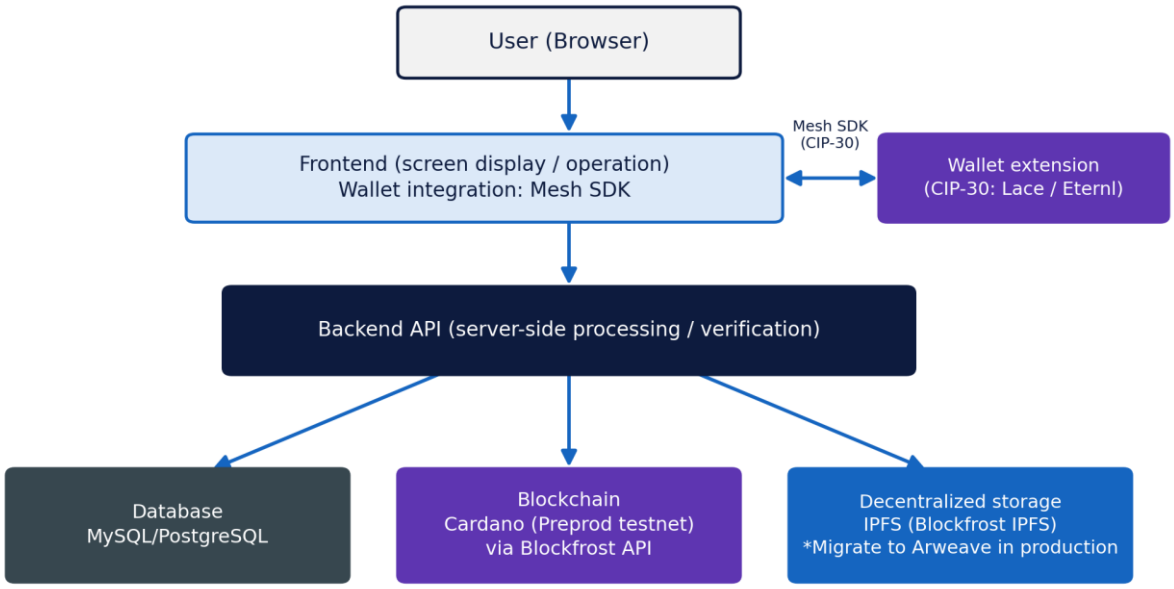
Role	Example Users	Main Permissions
Group Manager (GROUP_OWNER)	Teacher, organizer, administrator	Create a group, edit its name, manage members, change roles, give final approval to finalize a capsule
Capsule Manager (CAPSULE_OWNER)	Student representative, person in charge	Create a capsule, change settings while it is being edited, approve or reject registered content, seal it, and issue the certificate (NFT)
Member (MEMBER)	General member, student	Register to their own slot, edit content while it is being edited, view content after opening

Table 2-3 Three-Role Permission Model

3. System Configuration (Corresponding to: C1-1)

The system configuration for realizing the service concept above is shown below. The screen (frontend) communicates directly with the wallet extension, used to obtain the address and signature at connection time. The server (backend), meanwhile, queries the blockchain to verify recorded content — giving the system two distinct paths.

In this prototype, for important processing such as verifying records or issuing certificates, having the backend query the blockchain directly (on-chain verification) is adopted as the basic policy. This ensures the system always operates based on the latest recorded content on the blockchain, rather than relying solely on information held on the screen side.



* The wallet extension communicates directly with the frontend, used to obtain the address and signature at connection time.

In this prototype, the backend querying and verifying the blockchain directly is adopted as the basic approach.

Figure 3-1 Overall System Architecture

3.1 Technology Stack

Layer	Technology	Purpose / Notes
Frontend	Screen display mechanism (HTML, CSS, JavaScript) Wallet integration: Mesh SDK	Displays the operating screens in the browser, and handles connecting to the wallet and encrypting registered content.
Backend	Node.js / Express.js	The server-side mechanism that receives requests from the screens and handles data storage and blockchain integration.
Blockchain	Cardano (test environment)	Issues the certificate (NFT) that records content and guarantees it has not been changed. Switched to the production environment at launch.
Decentralized Storage	IPFS (Blockfrost IPFS)	Where encrypted text and images are stored. Adopted as the prototype's specification; migrated to Arweave in production (see A1).
Database	MySQL/PostgreSQL	Manages information about capsules and groups.

Table 3-1 Technology Stack

(reflecting the A1 research findings; decentralized storage adopts IPFS as the prototype specification)

3.2 Cardano-Specific Technology Stack

The technology stack described in 3.1 centers on the components of a conventional web application: the frontend, the backend, and the database. However, what this prototype development sets out to demonstrate is the practical usability of Cardano itself. This section therefore organizes how each Cardano-specific technical element will be used and what each one achieves, together with the implementation policy for it. Note that this document is an output of Milestone 1 (Prototype research and basic specifications), so what is described here defines the policy to be implemented in the prototype development that follows. Note also that smart contracts written in Aiken are to be adopted at the full development stage, and are out of scope for this prototype development.

How Records Are Made On-Chain (the Certificate and Its Contents)

ChainCapsule issues a certificate (an NFT) using Cardano's native asset functionality. This certificate records, as CIP-25 format metadata, information such as the storage location of the encrypted content (content_cid) and the planned opening date and time (unlock_time); see A1 for details. Once issued, this certificate is inscribed on the Cardano ledger semi-permanently, in a form that no one can rewrite. Recording the promise itself — what is kept, until when, and as whose record — onto the Cardano blockchain is precisely the core reason this project adopts Cardano.

How the Chain Is Queried (Blockfrost API)

To query the Preprod testnet ledger (address balances, certificate ownership, transaction status) without operating a Cardano node of our own, the Blockfrost API is used. The path described at the beginning of Chapter 3, in which the backend queries the blockchain directly to verify records (on-chain verification), is realized in implementation terms through these calls to the Blockfrost API. Blockfrost also provides an IPFS service, which the prototype uses as its storage provider (see 3.1). This allows on-chain queries and content storage to be handled through a single Cardano infrastructure provider, and pinned content can optionally be backed up to the Filecoin network for long-term retention until the move to Arweave in production.

How the Wallet Is Integrated (Mesh SDK)

The frontend connects to the user's wallet (Lace, Eternl, and other CIP-30-compliant wallets) through the Mesh SDK, and builds, signs, and submits the certificate issuance (Mint) transaction. The Mesh SDK can also load the compiled output of a smart contract written in Aiken (plutus.json), but that capability is used at the full development stage described below, and is not used in the prototype development.

Smart Contract Layer (Aiken) — Adopted in Full Development, Out of Scope for the Prototype

Aiken is a language for writing smart contracts on Cardano. As the mechanism that makes the blockchain itself enforce the opening conditions, the policy is to adopt it at the full development stage, for the production environment (decision D-03).

For this prototype development, however, Aiken is out of scope. The purpose of the prototype development is to confirm that the sequence from wallet connection through to certificate (NFT) issuance holds together, so the judgment of the opening conditions is first implemented on the Node.js side and verified there. Replacing it with an Aiken validator will be done after the prototype development is complete, at the point where full development begins.

Even with this phased approach, the issuance of the certificate (NFT) is by design carried out on-chain using Cardano's native asset functionality. The core element — recording to the Cardano blockchain — therefore holds even at the prototype stage. What remains on the Web2 side is only part of the verification logic, namely the judgment of the opening conditions. Replacing that with Aiken in full development moves the design to one in which Cardano itself guarantees, without relying on any trusted third party, that no one — including the operator — can open a capsule unless the opening conditions are met.

Why the Testnet Was Chosen

Using the Preprod testnet allows the same functionality as production (native assets, EUTXO, and the smart contract execution environment) to be verified with free test ADA obtained from a faucet, rather than with real ADA. This keeps the cost and risk of trial and error during development low, while still confirming technical behavior that is all but identical to production.

What This Means for the General User

Users can verify their identity with a wallet they already hold, without creating a new password. When a capsule is finalized, a certificate (NFT) is actually issued on the Cardano ledger, and anyone can confirm that issuance record on a public explorer. This means the claim "it has been recorded on the blockchain" is delivered as a concrete experience that users can verify for themselves. In addition, Cardano's transaction fees are low and predictable compared with other major blockchains, so records can be left without users having to worry about high fees. Together, these turn the abstract "value of blockchain" into an intuitively understandable experience: that one's own record is, without doubt, inscribed on a public ledger in a form that cannot be altered. This corresponds directly to

the goal this prototype development aims at, namely demonstrating the practical usability of Cardano to the general public.

Cardano Element	Role	Implementation Policy for the Prototype	Value for the General User
Certificate (NFT)	Records the promise in a form that cannot be altered	To be implemented (CIP-25, selected in A1)	Visible evidence that anyone can verify on a public explorer
Blockfrost API	Queries on-chain information (balance, ownership, TX status) and stores encrypted content via Blockfrost IPFS	To be implemented	Behind-the-scenes technology that underpins the reliability of verification
Mesh SDK	Wallet connection and transaction construction	To be implemented	Delivers a login experience that needs no password
Aiken (smart contract)	On-chain enforcement of the opening conditions	Out of scope for the prototype. Adopted in full development (D-03). During the prototype, the judgment is handled on the Node.js side instead	Realizes a design in which, in future, not even the operator can open a capsule
Preprod testnet	Verification in an environment equivalent to production, at no cost	To be adopted	Reduces development cost and risk

Table 3-2 Cardano-Specific Technical Elements and the Implementation Policy for the Prototype Development

3.3 Key Points of the Data Model

The database is centered around five groupings of information: groups, group members, capsules, the registration unit within a capsule (pieces), and viewing permissions. Both individual and group capsules are always managed uniformly as belonging to some group. In addition, the right to open a capsule is determined not by directly designating a specific individual, but by whether or not the certificate (NFT) issued at sealing time is held.

4. Linking Blockchain Functions and the User Interface (Corresponding to: C1-2)

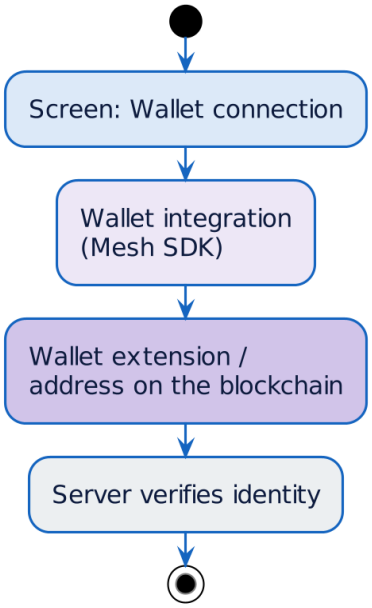
To make blockchain functionality usable through a "minimal user interface," this chapter organizes the ways UI operations link to the blockchain into the following 5 patterns. These 5 patterns correspond to the key user operations organized in B1 (Connect, View Data Information, Register Data, Upload, and Manage Access); "NFT Generation," which B1 treats as an independent item, is positioned in this document as the final step in the lifecycle of ④ "Register Data" (the certificate-issuance process performed at sealing time).

No	Linkage	UI Operation (Screen)	Related Blockchain Function	Linkage Mechanism
①	Connect	Wallet connection screen (SCREEN-01 / 02)	Obtain the wallet address	Wallet integration (Mesh SDK)
②	Upload	Piece registration screen (SCREEN-06)	Save the encrypted content to decentralized storage	Storage service (Blockfrost IPFS)
③	View Data Information	Dashboard, etc. (SCREEN-03 / 05 / 10)	Check and display recorded content	Blockchain query
④	Register Data	Capsule finalization screen (SCREEN-08)	Issue the certificate (NFT) (sealing process)	Certificate issuance mechanism (Mesh SDK)
⑤	Manage Access	Group management screen (SCREEN-11)	Determine opening rights by checking certificate (NFT) ownership	Blockchain query

Table 4-1 Five Ways Blockchain Functions and the UI Are Linked (Overview)

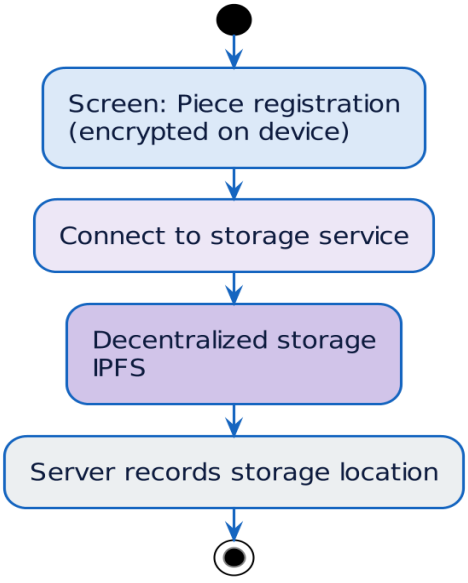
4.1 Connect

When a user approves the connection to a wallet app on the wallet connection screen, the wallet returns an address and a signature to the server. The server uses this information to verify identity and put the account into a usable state.



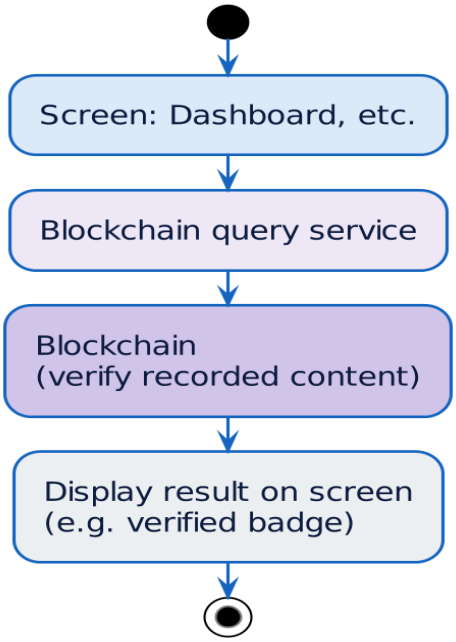
4.2 Upload

Text or images selected on the piece registration screen are encrypted on the user's device before being sent. The encrypted content is uploaded to decentralized storage (IPFS), and information about where it was stored is recorded on the server. This storage-location information is also recorded in the certificate (NFT) at the time of sealing.



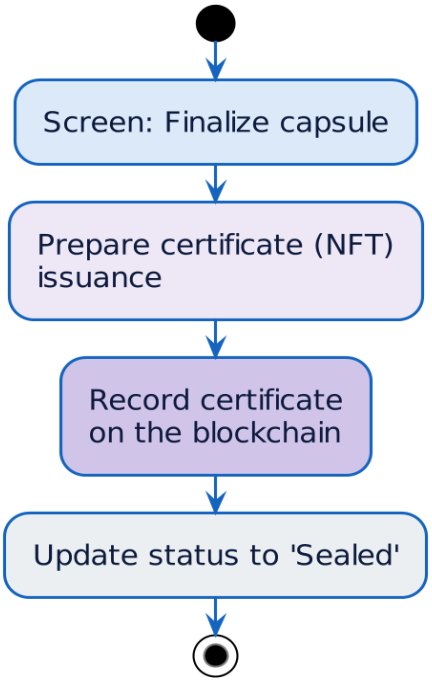
4.3 View Data Information

Screens such as the dashboard check content recorded on the blockchain via the server, and reflect it on screen as a display indicating that it has been verified.



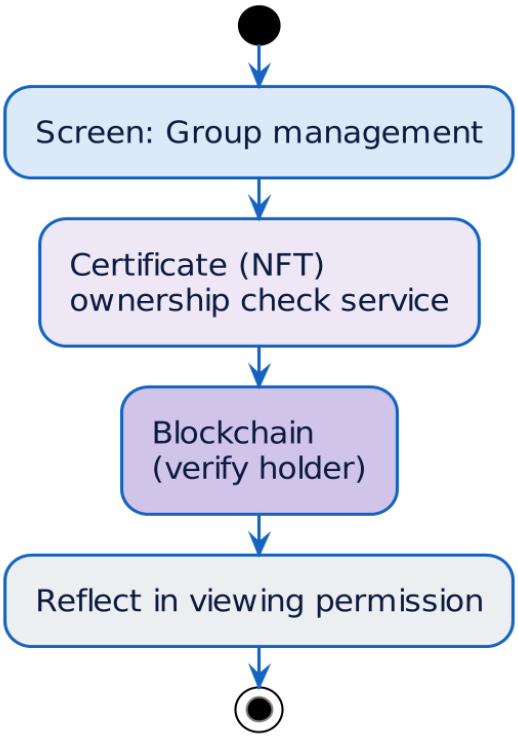
4.4 Register Data (Including Certificate Issuance)

When the capsule's manager finalizes registered content, the server prepares to issue a certificate (NFT), which is then recorded on the blockchain once the manager signs with their wallet. As a result, the capsule's status is updated to "Sealed."



4.5 Manage Access

When editing permissions on the group management screen, or when determining who may view content after opening, the blockchain is queried to check who holds the corresponding certificate (NFT), and the result is reflected in the viewing permissions. The specific way a group capsule is opened (its opening mode) is explained in detail in Chapter 5.



5. Group Capsule Opening Modes (D-11)

Because a group capsule is designed so that a certificate (NFT) is issued for each piece, it was necessary to define, in an integrated way, who can open the capsule as a whole. This was recorded as the project's open issue T-01 (priority: high). This chapter organizes the decision made in response to that issue (Decision Log D-11); with the content of this chapter, T-01 is now considered resolved.

5.1 The Three Opening Modes

When creating a capsule, the group's manager selects one of the following three opening modes. Once selected and the capsule is sealed, the mode cannot be changed. This is consistent with the service's overarching principle that a finalized record is never altered.

Mode	Condition for the Registrant to Open	Condition for a Manager to Open	Intended Use
①	Holds the certificate (NFT) for their own piece + the opening date/time has arrived	(no manager path exists)	Close to individual-style use; each piece can only be opened by the person who registered it
②	Same as above	Holds the mass-unlock certificate + the opening date/time has arrived	Use cases such as a graduation ceremony or reunion, where everyone views the content together on the day
③	Same as above	Holds the mass-unlock certificate (no time condition, but a condition on the number of managers in agreement applies)	An exceptional case intended only for emergencies

Table 5-1 The Three Opening Modes

Regardless of the mode, the basic path — "the person who registered a piece can open and view that piece once its planned opening date/time has arrived" — remains common and

unchanged. What differs by mode is only whether an additional manager path exists, and whether that path carries a time restriction.

5.2 Extending the Data Model

To realize this mechanism, each piece's lock information (its Datum) is extended to hold the following fields.

Field	Description
Piece certificate identifier	Existing field. Identifies the certificate (NFT) specific to that piece.
Planned opening date/time	Existing field. In principle, no one can open the piece until this date/time is reached.
Opening mode	New field. One of ①, ②, or ③. Selected by the group manager at capsule creation time.
Mass-unlock certificate identifier	New field. Set only for modes ② and ③. Identifies the shared certificate distributed to the group's managers.
List of managers	New field. A list identifying the managers who hold the mass-unlock certificate.
Emergency-opening agreement threshold	New field. The number of managers whose agreement is required to execute mode ③.

Table 5-2 Fields Held in a Piece's Lock Information (Datum)

For a mode-① capsule, no mass-unlock certificate is issued at all, so that no unused certificate is left in a manager's hands. For mode-② and mode-③ capsules, the certificate-issuance (sealing) transaction issues both the per-piece certificates and a mass-unlock certificate for each manager together, sending them to each manager's wallet.

5.3 How an Opening Is Decided

The opening transaction for each individual piece is judged according to the following logic.

Subject of the decision: the opening transaction for a single piece

The registrant's path:
The signer holds the certificate specific to that piece, AND
The current time is past the planned opening date/time
→ If satisfied, allow

The manager's path (exists only when the opening mode is ② or ③):
The signer holds the mass-unlock certificate, AND
If the mode is ②: the current time is past the planned opening date/time → if satisfied, allow
If the mode is ③: the number of managers in agreement is at or above the required threshold → if satisfied, allow

If none of the above conditions are met, do not allow the opening

Figure 5-1 Logic for Deciding Whether to Allow an Opening

5.4 Protecting Mode ③ (Emergency Opening)

Mode ③ is an exceptional capability that allows a manager to open a capsule without waiting for its planned opening date/time. Because it breaks, for managers only, the service's central promise that "no one can open a capsule before the specified date," two safeguards are put in place so that it can never be executed by a single manager's decision alone.

- **Mode ③ can only be selected for a capsule with 2 or more registered managers (if there is only 1 manager, mode ③ is removed from the available choices entirely)**
- **Executing mode ③ requires the simultaneous agreement (signatures) of a predetermined number of managers**

By default, the required number of agreeing managers is a strict majority of the registered managers (N). As a special case, however, when there are exactly 2 managers, the agreement of just 1 is sufficient (because a strict majority of 2 would otherwise require both managers, which is effectively the same as unanimous agreement). At capsule creation, a fixed number or unanimous agreement can also be specified individually.

Registered Managers (N)	Agreement Required	Notes
1	—	Mode ③ cannot be selected
2	1 (special case)	A special rule that departs from the strict-majority principle
3	2	Standard majority calculation
4	3	
5	3	
6	4	
7	4	
8	5	
9	5	
10	6	

Table 5-3 Required Agreement Under the Default (Majority) Rule

Operationally, opening under mode ③ follows the flow: a manager requests the opening, the request is sent to the other managers for approval, and the opening becomes executable once the required number of approvals has been gathered. This mirrors the same idea as Requirement A-02's forced sealing (where the CAPSULE_OWNER can force a capsule to be sealed once a set agreement threshold is reached); rather than introducing a new kind of operation, this simply applies the existing agreement mechanism to the opening step as well.

6. Summary

This document outlines the basic specifications required for the MIRAI BANK TIME Capsule (ChainCapsule) prototype, designated as Deliverable C1 of Milestone 1. First, the service's objective was defined as "accurately recording people's thoughts for the future," and a four-layer conceptual model—comprising individual users, groups, capsules, and pieces—was established. Next, the system architecture was clarified, integrating wallet authentication, encryption, decentralized storage, certificate (NFT) issuance, and on-chain verification. By delineating the roles of the frontend, backend, Cardano blockchain, IPFS, and database, the scope of implementation for the prototype was concretely defined. Furthermore, the interaction between blockchain functions and the user interface was categorized into five processes: connection, upload, data display, data registration, and access management. This configuration enables users to perform minimal operations while ensuring the authenticity of records and the validity of opening rights are verified in the background. Additionally, regarding the previously unresolved issue of consolidating opening rights for group capsules, three opening modes and a mechanism for administrator consensus were defined. Safeguards were specifically implemented for "emergency opening" to prevent execution based solely on the judgment of a single administrator. In summary, the system architecture (C1-1) and the integration methods between the blockchain and UI (C1-2) have been fully outlined in this document. The status of these items is shown in Table 6-1.

Acceptance Criterion	How It Is Addressed	Notes
C1-1 System Configuration	Clarified the overall system architecture, technology stack, and key points of the data model, based on the service concept and four-layer conceptual model	Decentralized storage unified to IPFS as the prototype specification. On-chain verification path stated as the basic policy
C1-2 Blockchain / UI Linkage	Organized 5 linkage patterns: Connect, Upload, View Data Information, Register Data (including certificate issuance), and Manage Access	B1's NFT Generation is integrated into ④ Register Data. Each pattern is organized together with a data flow diagram
(Reference) Resolution of T-01	Organized group capsule opening modes ①/②/③, and the manager-agreement mechanism protecting mode ③ (D-11)	Not an acceptance criterion; organized in Chapter 5 as the response to the project's open issue T-01

Table 6-1 Status of Response to the Acceptance Criteria